

字典和集合

主要内容

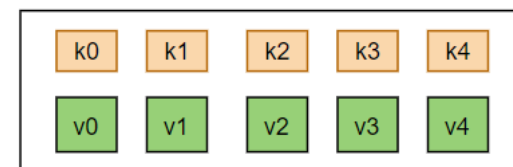
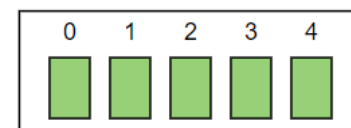
- 字典的定义和字典映射解包
- 字典的方法
- sorted函数
- 集合

字典(dictionary)

- 列表、元组等有序对象按顺序存储多个对象（元素），这些对象可通过**整数下标**快速访问

- 字典是包含**键-值对(key-value pair)**的映射(map)类型的**可变**序列：key映射到一个value

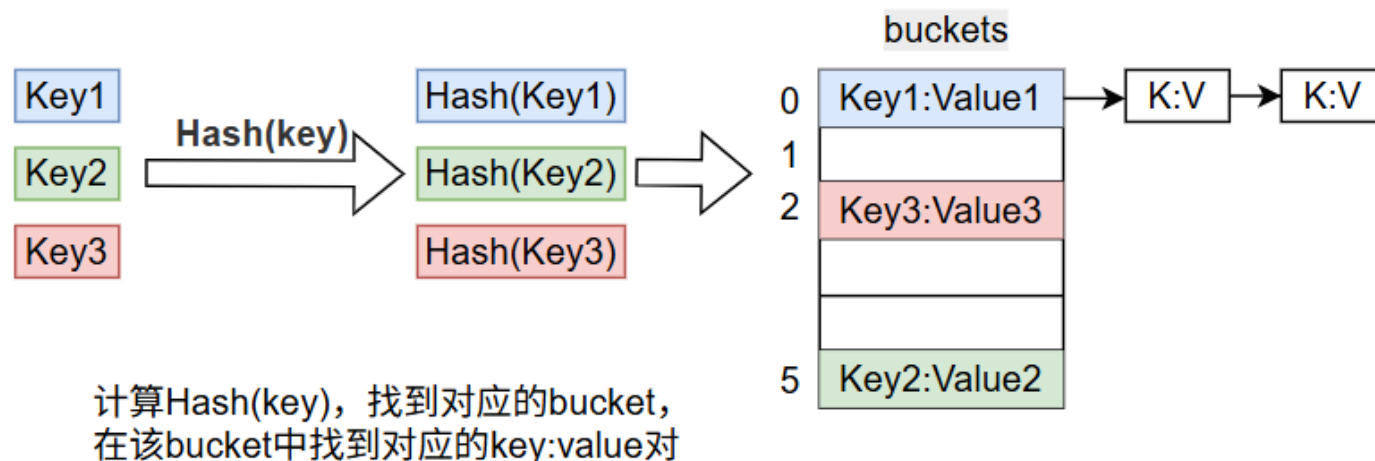
- 字典中每个元素包含两部分：**key和value**。添加元素时必须同时指定key和其对应的value
 - 字典中每个元素包含两部分：**key和value**。添加元素时必须同时指定key和其对应的value
- 值(values)可是任何对象
- 键(keys)不允许重复。一个key只能映射为某个value
 - key并不要求是同种类型
 - key必须是可hashable对象，即可调用hash(obj)得到一个值
- 通过key可以很快地找到其映射的value: dictobj[key]
- 已知对象(value)要找到其对应的key，需要许多额外的工作(要通过遍历寻找)



```
s[0]  
s[0] = 1  
s.append(5)  
s.extend(...)  
s.insert(...)  
del s[i]  
  
d['one']  
d['one'] = 1  
d.update(...)  
del d[key]
```

key的值不允许通过任何手段修改

- 整数、实数、复数、字符串等不可变对象可以作为key
- 不可变的容器对象(如元组) 有可能可以作为key
- 可变对象不能作为key



创建字典1: 字典字面量(dict literal)

{key1:value1 ,key2:value2, ... ,keyN:valueN}

- 大括号界定, 元素用key:value表示, 元素间用逗号分隔
- 按照顺序将key:value加入字典
- **构造字典(包括后续的其他类似方法) 时允许key重复。** 如果后面的元素的key与之前的一致, 则更新其对应的值
- Python3.7: python语言明确字典记录插入 (注意不是更新的) 的顺序, 遍历时按照**key插入到字典中的先后顺序**访问
- 对于python3.7之前的版本, 如果要求记录插入顺序, 应该采用 collections.OrderedDict

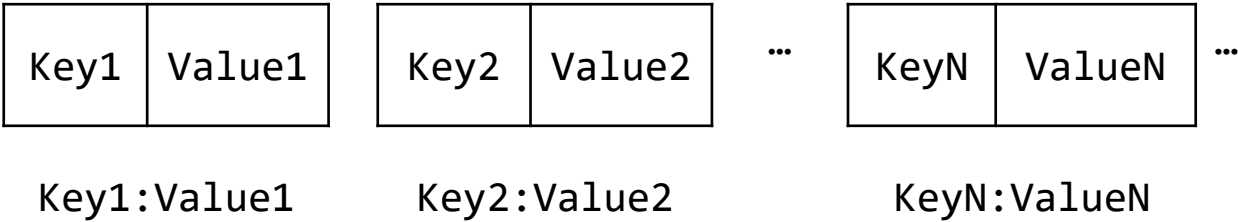
```
>>> d = {} # 空字典,长度为0
>>> d1 = {"one": -1, "two": 2, "three": 3, "one": 1}
# 插入顺序: 'one', 'two', 'three'
# 更新顺序: 'two', 'three', 'one'
>>> d1
{'one': 1, 'two': 2, 'three': 3}
>>> d2 = {
    "name": "Steve",
    "age": 25,
    "gender": "male",
    "address": {"city": "shanghai", "zip": "200433"},
}
>>> d3 = {
    0: '整数0',
    3.14: 'pi',
    (10, 1): '国庆节',
}
>>> {(1,[2]): 0} # key必须是hashable对象
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
```

创建字典2: 构造函数dict(...)

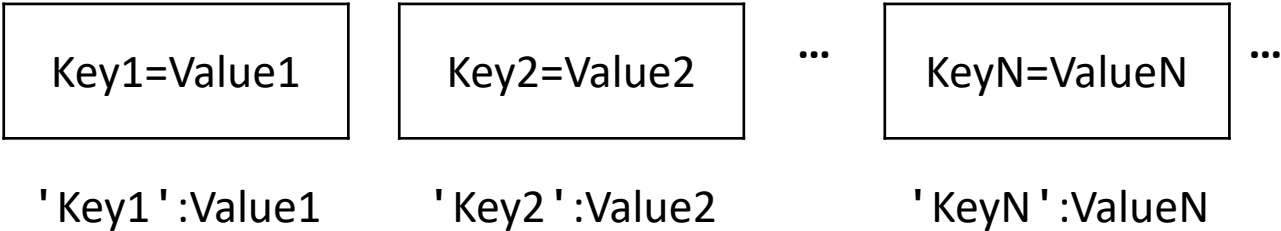
dict()	创建空字典
dict(mapping)	已有映射（字典）对象，创建一个新的字典（原有字典对象的copy）
dict(iterable)	新字典的元素来自于iterable，iterable中的 每个元素必须是一个包含2个子元素的容器对象 。第一个子元素作为key，第二个子元素为value
dict(**kwargs)	可变长关键字参数 ：收集函数调用时传递的 关键字参数(keyvar=value) ，创建一个新的字典。key为字符串类型，该字符串保存了关键字参数传递给出的变量名，而对应的value为关键字参数传递的值，即对应的键值对为 'keyvar':value

```
d1 = {'one': 1, 'two': 2, 'three': 3}
d2 = dict(d1)
d3 = dict(['one',1],('two',2),['three',3])
z = zip(['one', 'two', 'three'], [1, 2, 3])
d4 = dict(z)
d5 = dict(one=1, two=2, three=3)
d6 = d1.copy() # 等价于 dict(d1)
```

dict(iterable)



dict(Key1=Value1, Key2=Value2,...)

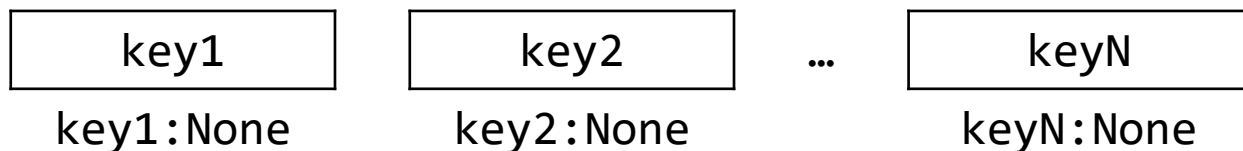
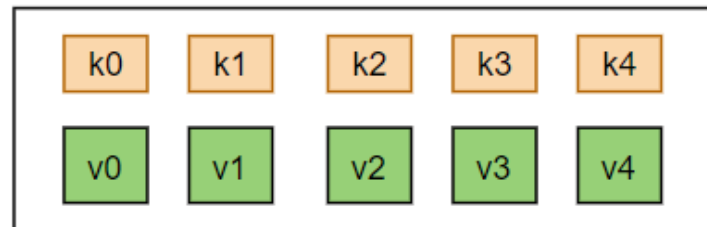


- 字典对象也支持copy方法，得到一个新的dict对象

创建字典3：类方法fromkeys

`dict.fromkeys(iterable[,value])`

- 每个元素的key来自于可迭代对象，值设置为value，缺省为None



- 实例方法(instance method): 首先创建一个对象，然后通过该对象来调用方法
- 静态方法(static method): `str.maketrans(x, y[, z])`, `str.maketrans(mapping)`
- 类方法 (class method): 从调用者的角度，静态和类方法没有区别
 - 通过类名调用方法，无需首先创建对象的实例
 - 当然也可创建对象实例，然后再调用类方法

```
>>> d1 = dict.fromkeys(['name', 'age', 'gender'])
>>> d1
{'name': None, 'age': None, 'gender': None}
>>> d2 = dict.fromkeys(range(5), 10)
>>> d2
{0: 10, 1: 10, 2: 10, 3: 10, 4: 10}
```

创建字典4：字典解析式 (6.5.3)

- 列表解析式: [expr for value in iterable if condition]
- 生成表达式: (expr for value in iterable if condition)
- **字典解析式**: {key:val for value in iterable if condition}

```
d = {c: c * 4 for c in 'abcd'}  
# {'a': 'aaaa', 'b': 'bbbb', 'c': 'cccc', 'd': 'dddd'}  
  
scores = [['Alice', 90], ['Bob', 80], ['Carol', 70], ['Tom', 45]]  
score_dict = dict(scores)  
# score_dict = {name: score for name, score in scores}  
# {'Alice': 90, 'Bob': 80, 'Carol': 70, 'Tom': 45}  
  
pass_dict = {name: score for name, score in scores if score >= 60}  
# {'Alice': 90, 'Bob': 80, 'Carol': 70}
```

映射解包

- 序列解包(iterable unpacking)可用在:

- 赋值语句 `x, y, z = 1, 2, 3`; `x, *y, z = range(10)`
- 函数调用: `print(*[1, 2, 3])`
- 列表、元组字面量定义中: `[*[1, 2, 3], 4, 5]`

- 映射解包(mapping unpacking)

- 函数调用: `func(..., **dictobj)`
 - dictobj的key必须为字符串类型
 - 相当于传递多个关键字参数, 每个关键字参数的名称为字典对象的元素的关键字(字符串)包含的名字(要满足变量名的要求), 而关键字参数的值为字典对象中key对应的值
- 字典字面量定义: `**dictobj`表示将该字典展开成键值对的形式

```
d1 = dict(one=1, two=2, three=3)
d2 = dict(**d1)
d3 = dict(**{'one':1, 'two':2}, three=3)
d4 = {**d1, 'four':4}
```

'Key1':Value1

'Key2':Value2

'KeyN':ValueN

Key1=Value1	Key2=Value2	...	KeyN=ValueN	...
-------------	-------------	-----	-------------	-----

```
d = {'k0':v0, 'k1':v1, 'k2':v2}
func(x, y, **d)
# 相当于func(x, y, k0=v0, k1=v1, k2=v2)

d = {k0:v0, k1:v1, k2:v2}
{'one':1, **d}
# 相当于 {'one':1, k0:v0, k1:v1, k2:v2}
```

```
one_liner_config = {'sep': '+', 'end': '='}
print(*range(1, 10), **one_liner_config)
# print(*range(1, 10), sep='+', end='=')
print(sum(range(1, 10)))
```

1+2+3+4+5+6+7+8+9=45

创建字典（总结）

字面量定义中的映射解包	<pre>{'name': 'Steve', 'age': 25, 'sex': 'male'} {**{'one': 1, 'two': 2, 'three': 3}, 'four': 4}</pre>
- dict() - dict(iterable) - dict(**kwargs) - dict(mapping)	- 空字典 - 迭代对象中的元素必须是包括两个对象的容器对象，比如dict([('name', 'Steve'), ('age', 25), ('sex', 'male')]) - 可变长关键字参数：字典的key为字符串类型，包含关键字参数的名字，比如dict(name='Steve', age=25, sex='male') - 类似于原有mapping(即dict)的拷贝
dict.fromkeys(iterable[,value])	iterable中的元素作为字典中元素的key，而值为第二个参数或缺省的None，比如dict.fromkeys(('name', 'age', 'sex'), None)
{k:v for k,v in ...}	{k:v for k,v in zip(('name', 'age', 'sex'), ('Steve', 25, 'male'))} 等价于 {'name': 'Steve', 'age': 25, 'sex': 'male'}

主要内容

- 字典的定义和字典映射解包
- **字典的方法**
- sorted函数
- 集合

字典支持的操作

已知key,是否有key:value对? 对应的value是什么

遍历

插入或修改

删除

方法	说明
key in d key not in d	判断字典d中有没有键==key, 返回True or False 相当于 key in d.keys()
d[key]	返回key对应的value, key不存在raise KeyError
d.get(key[,default])	返回key对应的value, key不存在时返回default, 缺省为None
d.keys()	返回可迭代对象, 给出了字典中的所有key的视图(view)
d.values()	返回可迭代对象, 给出了字典中的所有value的视图
d.items()	返回可迭代对象, 给出了字典中key-value对的视图, 其每个元素为元组(key,value)
d[key] = value	更新字典中键=key的元素的值为value, 如果不存在, 则添加key:value
d.setdefault(key[,default])	如果key不在字典, 插入新元素, 其值为default(缺省None)。如果在, 不更新。返回d[key]
d.update(other)	根据另一字典或元素为key/value对的可迭代对象更新字典, 返回None
del d[key]	删除元素, 如果key不存在, raise KeyError
d.pop(key)	如果key存在删除对应元素并返回值, 否则raise KeyError
d.pop(key,value)	如果key存在删除对应元素并返回值, 否则返回value
d.popitem()	移走并返回某一个(key,value) 对。如果空字典, raise KeyError
d.clear()	清除所有元素
d.copy()	返回shallow copy后的新字典



字典方法：成员关系判断

key in dictobj和 key not in dictobj:

- 判断字典对象中有没有键==key, 返回True or False

```
>>> d1 = {0: 'one', 1: 'two'}
>>> 0 in d1
True
>>> 'one' in d1
False
```

- 字典支持 == 和 != 运算, 判断key:value对是否完全一致, 先后顺序无关
- 不支持大小关系的比较, 不支持 + - 等运算

```
>>> d2 = {1: 'two', 0: 'one'}
>>> d1 == d2
True
>>> d1 > d2
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    d1 > d2
```

TypeError: '>' not supported between instances of 'dict' and 'dict'

字典方法：修改字典

d[key]=value 新增或更新key:value对

- 如果key存在，则**更新**字典中键=key的元素的值为value；如果key不存在，则**添加**key:value

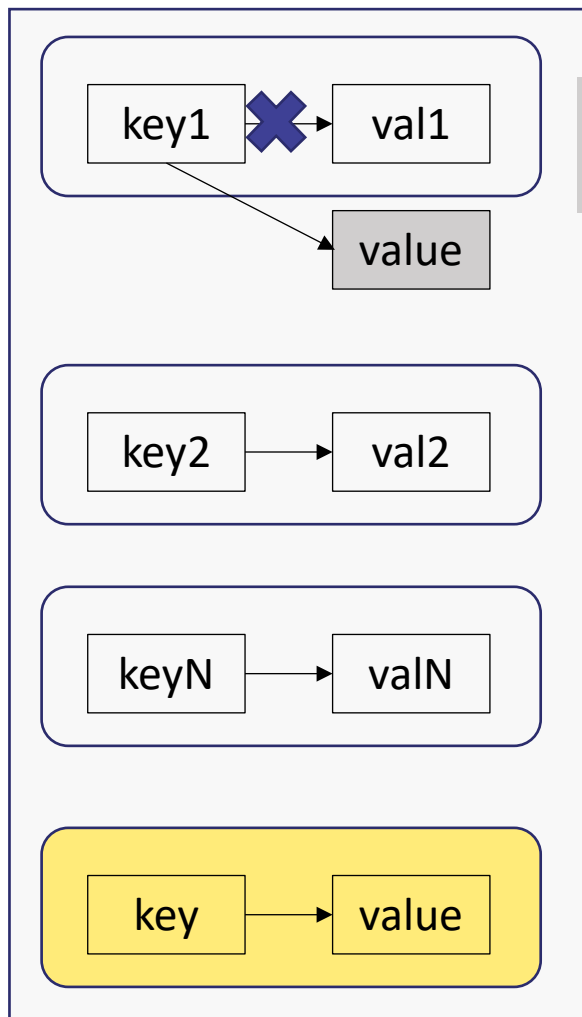
dictobj.update(another) another为映射或iterable对象

dictobj.update(key1=value1, key2=value2)

- update(.) 新增或更新多个key:value，其参数与dict()的参数类似
 - 字典another: 将another中的key:value对加入到当前字典对象
 - 可迭代对象another: 要求其元素为包含两个元素的容器对象
 - 采用关键字参数传递：key=value

```
>>> d = {'name': 'Tony', 'gender': 'male', 'age': 37}
>>> d['age'] = 38
>>> d['city'] = 'BeiJing'
>>> d.update({'age': 39, 'city': 'Shanghai'}) # 更新多个key:value
>>> d
{'name': 'Tony', 'gender': 'male', 'age': 39, 'city': 'Shanghai'}
>>> d.update([('age', 40), ('phone', '1234')])
>>> d.update(age=41) # 采用关键字参数传输
>>> d
{'name': 'Tony', 'gender': 'male', 'age': 41, 'city': 'Shanghai', 'phone': '1234'}
```

```
for key in another: # map对象
    dictobj[key] = another[key]
for key, value in another: # 可迭代对象
    dictobj[key] = value
```



更新key:value
d[key1] = value

添加key:value
d[key] = value

字典方法：访问字典元素

dictobj[key] dictobj.get(key, default=None, /)

- d[key]: 返回字典中key对应的value; key不存在时, 抛出KeyError异常
- d.get(key, default=None, /): 返回字典中key对应的value; key不存在时返回default, 缺省为None

```
# get(key, default=None)
if key in d:
    return d[key]
else:
    return default
```

```
>>> d = {'name': 'Tony', 'gender': 'male',
'age': 37}
>>> d['name']
'Tony'
>>> d['city']
Traceback (most recent call last):
  File "<pyshell#53>", line 1, in <module>
    d['city']
KeyError: 'city'
```

```
>>> d.get('gender', 'N/A') # key存在
'male'
```

key不存在, 返回默认值

```
>>> d.get('city') # return None
>>> d.get('city', 'N/A')
'N/A'
```

```
if 'gender' in d :
    gender = '性别:' + d['gender']
else:
    gender = '性别: 未知'

try:
    gender = '性别:' + d['gender']
except KeyError:
    gender = '性别: 未知'

gender = f'性别: {d.get("gender", "未知")}'
```

字典方法：访问元素与修改元素结合

`get(key, default=None, /)` 返回`d[key]`或者`default`

`setdefault(key, default=None, /)` 如果key不在字典，会首先添加key:default。返回`d[key]`

- 如果key在字典里，则返回`d[key]`。
- 相比`get(key[,default])`而言，如果key不在字典里时会首先初始化key对应的value为`default`，返回`d[key]`(即`default`)

```
# get(key[, default])
if key in d:
    return d[key]
else:
    return default
```

```
# setdefault(key[, default])
if key in d:
    return d[key]
else:
    d[key] = default
    return d[key]
```

```
>>> d
{'name': 'Tony', 'gender': 'male', 'age': 41, 'city': 'Shanghai', 'phone': '1234'}
>>> d.setdefault('age')
41
>>> d.setdefault('photo')
>>> d
{'name': 'Tony', 'gender': 'male', 'age': 41, 'city': 'Shanghai', 'phone': '1234', 'photo': None}
>>> d.setdefault('office', '201')
'201'
>>> d
{'name': 'Tony', 'gender': 'male', 'age': 41, 'city': 'Shanghai', 'phone': '1234', 'photo': None, 'office': '201'}
```

拓展: defaultdict

模块collections中引入了一个特别的字典类型defaultdict

- defaultdict(factory[,...])用于创建一个defaultdict对象
 - 第一个参数为一个**可调用对象factory** (函数), 其余参数与dict函数的参数一样
 - 字典的key:value对的value缺省值为 **调用factory()所返回的结果**
- dict的所有方法都可用于defaultdict
- 在通过dictobj[key]访问元素时:
 - 如果key存在, 返回dictobj[key]
 - 如果key不存在, 会首先设置dictobj[key]为一个缺省值, 然后返回dictobj[key]
dictobj[key] = value = factory(); return value
 - 副作用是如果不小心写错了key, 会增加一个新的元素

```
# defaultdict: d[key]
if key in d:
    return d[key]
else:
    d[key] = factory()
    return d[key]
```

```
from collections import defaultdict
d = defaultdict(list, zip('abc', [[] for i in range(3)]))
d['a'].append(1)          # d = defaultdict(<class 'list'>, {'a': [1], 'b': [], 'c': [], 'd': [4]})
d['d'].append(4)

d2 = defaultdict(int)
d2['a'] = d2['a'] + 1      # d2 = defaultdict(<class 'int'>, {'a': 1})
```

字典方法：读取和修改

<code>d[key]</code>	返回key对应的value, key不存在raise KeyError
<code>d.get(key[,default])</code>	返回key对应的value, key不存在时返回default, 缺省为None
<code>d[key] = value</code>	更新字典中键=key的元素的值为value, 如果不存在, 则添加key:value
<code>setdefault(key[,default])</code>	如果key不在字典里插入新元素, 其值为default(缺省None)。如果在不更新。 返回d[key]
<code>update(other)</code> <code>update(key1=value1, key2=value2)</code>	根据另一字典或元素为key/value对的可迭代对象更新字典, 返回None 通过关键字参数传递key/value对
<code>d=defaultdict(factory)</code> <code>d[key]</code>	key如果不存在时, 会自动添加key:factory()到字典中, 即 <code>d[key] = factory()</code>

字典方法：删除元素

<code>del d[key]</code>	删除元素，如果key不存在， raise KeyError
<code>pop(key)</code>	如果key存在，删除对应元素并返回值，否则raise KeyError
<code>pop(key, default)</code>	如果key存在，删除对应元素并返回值，否则返回default
<code>popitem()</code>	移走并返回某一个(key,value) 对，如果为空 raise KeyError
<code>clear()</code>	清除所有元素

```
>>> d = {'name': 'Tony', 'gender': 'male',  
        'age': 41, 'city': 'Shanghai'}  
>>> del d['city']  
>>> d.pop('gender')  
'male'  
>>> d.pop('gender')  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
KeyError: 'gender'  
>>> d.pop('gender', 'N/A')  
'N/A'
```

```
print(f'{d = }')  
# {'name': 'Tony', 'age': 41}  
while d:  
    item = d.popitem() # 随机删除一个键值对  
    print(f'{item = }')
```

```
item = ('age', 41)  
item = ('name', 'Tony')
```

字典方法：遍历字典

- 遍历时如果对应的字典对象的**大小有改变时**会抛出异常RuntimeError

len(d)	返回字典中元素的个数
d.keys()	返回可迭代对象，给出了字典中的所有key的视图(view)
d.values()	返回可迭代对象，给出了字典中的所有value的视图
d.items()	返回可迭代对象，给出了字典中key-value对的视图，其每个元素为元组(key,value)
iter(d)	返回迭代器对象，其中 元素为字典中的所有key ，相当于iter(d.keys()) 意味着d作为可迭代对象时，其元素为字典的key

```
d = {'name': 'Tony', 'gender': 'male', 'age': 37}
```

```
for key in d.keys():  
    print(key)
```

```
for key in d:  
    print(key)
```

name
gender
age

```
for value in d.values():  
    print(value)
```

Tony
male
37

```
for item in d.items():  
    print(item)
```

('name', 'Tony')
('gender', 'male')
('age', 37)

```
for key,value in d.items():  
    print(key,value)
```

name Tony
gender male
age 37

```
>>> d = {'name': 'Tony', 'gender': 'male', 'age': 37}  
>>> d.items()  
dict_items([('name', 'Tony'), ('gender', 'male'), ('age', 37)])  
>>> list(d.items())  
[('name', 'Tony'), ('gender', 'male'), ('age', 37)]  
>>> list(d)  
['name', 'gender', 'age']
```

字典应用示例1：遍历字典

例6-6变种：某次考试成绩出来了，计算最高分、最低分、平均分，并输出**所有最低分和最高分同学**

```
def print_exam_stats(exam_results):
    scores = exam_results.values()
    highest = max(scores)
    lowest = min(scores)
    average = sum(scores) / len(scores)
    lowest_students = [name for name in exam_results if exam_results[name] == lowest]
    highest_students = [name for name, score in exam_results.items() if score == highest]
    print(f'最高分{highest}, 最低分{lowest}, 平均分{average:.2f}')
    print("获得最低成绩{}的学生有{}人： {}".format(lowest, len(lowest_students),
                                                    ", ".join(lowest_students)))
    print("获得最高成绩{}的学生有{}人： {}".format(highest, len(highest_students),
                                                    ", ".join(highest_students)))

if __name__ == "__main__":
    names = ["Alice", "Tom", "Jack", "Owen", "Mary", "John", "David", "Ashley", "Emma", "Jessica", "Julia"]
    scores = [100, 82, 80, 57, 93, 100, 91, 84, 68, 73, 78]
    exam_results = dict(zip(names, scores))
    print_exam_stats(exam_results)
```

- for key in d: print(key, d[key])
- for k,v in d.items(): print(k, v)
- for item in d.items():
 print(item[0], item[1])

exam_results.py

字典应用示例2: value为不可变对象

首先生成包含1000个随机字符的字符串, 然后统计每个字符的出现次数。

- key为某个字符
- value为该字符出现的次数

`counters[item] = counters.setdefault(item, 0) + 1`
多赋值一次, 如果item不存在, 首先item:0, 然后item:1

```
def letter_counter(seq):  
    counters = dict() # counters = {}  
    for item in seq:  
        counters[item] = counters.get(item, 0) + 1  
    return counters
```

```
if item in counters:  
    counters[item] += 1  
else:  
    counters[item] = 1
```

letter_counter.py

```
def generate_text(length=1000):  
    import string  
    import random  
    valid_chars = string.ascii_letters + string.digits + string.punctuation  
    chars = [random.choice(valid_chars) for i in range(length)]  
    return ''.join(chars)  
  
def print_counter(counters):  
    for item in counters:  
        print('{}的出现次数==> {}'.format(item, counters[item]))
```

字典应用示例2: value为不可变对象

- 采用缺省字典: 字符的出现次数 缺省为 `int() = 0`

```
def letter_counter(seq):  
    counters = dict()    # counters = {}  
    for item in seq:  
        counters[item] = counters.get(item, 0) + 1  
    return counters
```

```
def letter_counter2(seq):  
    from collections import defaultdict  
    counters = defaultdict(int)  
    for item in seq:  
        counters[item] += 1  
    return counters
```

letter_counter.py

```
# defaultdict: d[key]  
if key in d:  
    return d[key]  
else:  
    d[key] = factory()  
    return d[key]
```

字典应用示例3: value为可变对象

字典对象d记录了多次考试的分数。key为某个同学的姓名，而对应的value为一个列表

假设某次考试成绩出来了，要保存名为name的同学的成绩score

```
name, score = 'tony', 98
```

- 如果以前有过成绩(key in d)，则首先获得原有成绩列表，将最新成绩原地附加到该列表
- 如果没有成绩，则创建一个包含本次成绩的新列表，并加入到字典中

```
exams = { }
```

```
if name in exams:
    exams[name].append(score)
else:
    exams[name] = [score]
```

```
exams.setdefault(name, []).append(score)
```

```
exams[name] = exams.get(name, [])
exams[name].append(score)
```

不建议的版本

```
exams[name] = exams.get(name, []) + [score]
```

update_exam_results.py

```
#.setdefault(key, default=None)
if key in d:
    return d[key]
else:
    d[key] = default
    return default
```

下述代码有错! append返回None, 等价于
d1_err[name] = None

```
d1_err = { }
```

```
d1_err[name] = d1_err.get(name, []).append(score)
```

使用加法运算的版本性能最差，每次都创建新的列表

字典应用示例3: value为可变对象

- 采用缺省字典: value为包括考试分数的列表, 缺省为 list() = []

```
exams.setdefault(name, []).append(score)
```

```
from collections import defaultdict
exams = defaultdict(list)

# name, score = 'tony', 98
exams[name].append(score)
```

```
# defaultdict: d[key]
if key in d:
    return d[key]
else:
    d[key] = factory()
    return d[key]
```

字典应用示例4：删除多个元素

删除字典中其值为value的所有key-value对

- for循环遍历字典时，循环体中增加或者删除导致**元素个数有变化时**会抛出异常RuntimeError

```
def remove_values0(d, value):  
    for key in d:  
        if d[key] == value:  
            del d[key]  
  
if __name__ == '__main__':  
    d = {'a': 1, 'b': 2, 'c': 3, 'd': 2}  
    remove_values0(d, 2)
```

...

for key in d:

RuntimeError: dictionary changed size during iteration

分解为两个步骤

- 首先获得要删除的元素的所有key
- 然后逐个删除

```
def remove_values(d, value):  
    keys = [key for key in d if d[key] == value]  
    for key in keys:  
        del d[key]
```

remove_values.py

可变长关键字参数与映射解包示例

- 函数调用：序列和映射解包将可迭代对象或可映射对象展开为位置参数或关键字参数
- 函数定义：可变长位置和关键字参数收集多余的位置参数或关键字参数组成元组或字典对象

```
def tracing(func, *args, **kwargs):  
    if hasattr(func, "calls"):  
        func.calls += 1  
    else:  
        func.calls = 1  
    print(f"[{func.calls}] calling function: {func.__name__}(", end="")  
    if args:  
        print(*args, sep=",", end=", " if kwargs else "")  
    if kwargs:  
        print(*(f"{k}={v}" for k, v in kwargs.items()), sep=",", end="")  
    print(")")  
    result = func(*args, **kwargs)  
    print(f"{func.__name__} returned: {result}")  
    return result
```

tracing.py

```
def demo(a, b=4, *x, **y):  
    print(a, b, x, y)
```

```
tracing(demo, 1)  
tracing(demo, 1, 2)  
tracing(demo, 1, 2, 3, 4, c=9, d=8)
```

如何记录函数调用次数的状态信息？

- 全局字典：key=函数对象, value=调用次数
- 函数对象的属性
 - `hasattr(obj, name)`: 对象是否有指定名字(name)的属性存在
 - `getattr(obj, name)`: 获得对象指定属性的值

```
[1] calling function: demo(1)  
1 4 () {}  
demo returned: None  
[2] calling function: demo(1,2)  
1 2 () {}  
demo returned: None  
[3] calling function:  
demo(1,2,3,4,c=9,d=8)  
1 2 (3, 4) {'c': 9, 'd': 8}  
demo returned: None
```

可变长关键字参数与映射解包示例2

函数parse_options:

(1) options给出了调用时传递的选项和相应的取值。

(2) 支持选项verbose/reverse/key, 如果没有传递时相当于: verbose=False, reverse=False, key=None

parse_kwargs.py

```
def parse_options(**kwargs):
    verbose = kwargs.pop('verbose', False)
    reverse = kwargs.pop('reverse', False)
    key = kwargs.pop('key', None)
    print(f'{verbose=}, {reverse=}, key={"function " + key.__name__ if key else None}')
    while len(kwargs):
        option, value = kwargs.popitem()
        print('unknown option:%s=%s' % (option, value))

if __name__ == '__main__':
    options = {'reverse':True, 'key':lambda item:item[0], 'option1':1, 'option2':2}
    parse_options(**options)
```

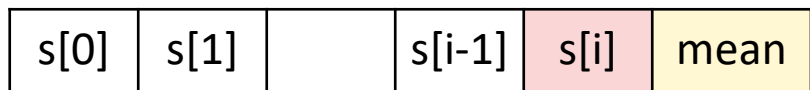
函数定义：传递的实参为可变对象

函数体内可以通过可变对象的相应方法原地修改

- 列表：
 - (列表对象支持的) 附加、删除
 - 通过列表对象的下标形式修改对应位置的元素
- 字典: d[key]=value; d.extend(...)

问题：更新时序序列的平均值

- s记录了时序序列以及目前的平均值
- value: 新的数值



mutable_update.py

```
import random

def mutable_update(s, value):
    average = (sum(s[:-1]) + value) / len(s)
    s[-1] = value
    s.append(average)

    # 下面为错误的版本
    # s = s[:-1] + [value, average]

if __name__ == '__main__':
    s = [0]
    for i in range(15):
        value = random.randint(1, 100)
        mutable_update(s, value)

    print(*s[:-1], sep=', ')
    print(f'平均值为: {s[-1]:.2f}')
```

函数定义：默认值参数可以为可变参数

arg=default: 形参arg没有传递实参时使用默认值default

- default在def语句构造函数对象时求解
- default求解后的结果作为属性__defaults__的元素
- 如果default求解后的对象为无法改变值的不可变对象，安全的方式
- 如果default为可改变值的对象（如可变对象）
 - 函数体可原地操作修改该对象
 - 可通过默认值对象保存状态信息
- 默认值为可变对象时可能带来bug，可改造为默认值为不可变对象

```
append_item.__defaults__ = ([],)  
append_item.__defaults__ = ([1],)  
append_item.__defaults__ = ([1, 1],)
```

```
def append_item(item, old_list=[]):  
    old_list.append(item)  
    return old_list  
  
def test_append_item():  
    items = append_item(5, [1, 2, 3, 4])  
    print(f'{append_item.__defaults__ = }')  
    items = append_item(1) # [1]  
    print(f'{append_item.__defaults__ = }')  
    items = append_item(1) # [1, 1]  
    print(f'{append_item.__defaults__ = }')  
  
def append_item2(item, old_list=None):  
    if old_list is None:  
        old_list = []  
    old_list.append(item)  
    return old_list
```

append_item.py

主要内容

- 字典的定义和字典序列解包
- 字典的方法
- **sorted函数**
- 集合

内置函数sorted

L.sort(key=None, reverse=False) 原地排序列表，返回None

sorted(iterable, key=None, reverse=False) 对可迭代对象的元素排序，返回新的列表

- reverse: 从小到大（缺省False），reverse=True表示从大到小排序
- key缺省为None，表示基于元素间的大小顺序排序
- key也可传递函数对象，基于调用该函数之后的排序基准进行排列
 - 在调用时传递一个参数，该参数为要排序的iterable对象中的某个元素
 - 函数调用的返回值作为给iterable对象中对应元素排序的基准
 - 排序算法是稳定的，在基准相同时按照原有出现的顺序排列

- 新列表的元素来自于可迭代对象
- 排序的基准：调用关键字参数key所传递的函数key(item)的返回值

10	5	23	20
----	---	----	----

$5 \leq 10 \leq 20 \leq 23$

5	10	20	23
---	----	----	----

```
s = [10, 5, 23, 20]
```

```
s1 = sorted(s)
```

```
print(s1)
```

e1	e2	e3	...						
----	----	----	-----	--	--	--	--	--	--

iterable



e _{i1}	e _{i2}	e _{i3}	...						
-----------------	-----------------	-----------------	-----	--	--	--	--	--	--

sorted(iterable): $e_{i1} \leq e_{i2} \leq e_{i3} \leq \dots$

e _{i1}	e _{i2}	e _{i3}	...						
-----------------	-----------------	-----------------	-----	--	--	--	--	--	--

sorted(iterable, key=key): $key(e_{i1}) \leq key(e_{i2}) \leq key(e_{i3}) \leq \dots$

内置函数sorted

- 传递函数对象，调用该函数（参数为待排序的元素）的结果作为排序的基准： `key(item)`
 - 可以通过def语句定义一个函数
 - 可以使用lambda表达式得到一个匿名函数对象
- 排序算法是稳定的，在基准相同时按照原有出现的顺序排列
 - 30和12的排序基准(都为3) 一样。在原来的列表s中，30出现在12之前

```
>>> s = [30, 26, 26, 15, 11, 1, 12, 4, 27, 11]
>>> s
[30, 26, 26, 15, 11, 1, 12, 4, 27, 11]
>>> sorted(s)
[1, 4, 11, 11, 12, 15, 26, 26, 27, 30]
>>> def sum_digits(x):
...     return sum(int(i) for i in str(x))
...
>>> sorted(s, key=sum_digits)
[1, 11, 11, 30, 12, 4, 15, 26, 26, 27]
>>> sorted(s, key=lambda x:sum(int(i) for i in str(x)))
[1, 11, 11, 30, 12, 4, 15, 26, 26, 27]
```

sum_digits(x): x转换为字符串后
获得每个字符再转换为整数，最
后求和，得到x的各个数字之和

sorted_demo.py

12345
↓
'12345'
sum(1 + 2 + 3 + 4 + 5)

内置函数sorted: 两个问题

`listobj.sort(key=None, reverse=False)` `sorted(iterable, key=None, reverse=False)`

- 对什么进行排序?
 - 列表的`sort`方法对于自身进行排序
 - 内置函数`sorted`返回新的列表, 新列表中的元素来自于可迭代对象
 - 传递的参数为元组、列表、集合等, 则新列表中的元素就是这些对象中原来的元素
 - 对于字典对象`dictobj`:
 - 传递的参数为`dictobj`, 新列表中的元素就是字典对象的`key`
 - 通过`key`可以得到其对应的`value`。 `sorted`传递的`key`参数可以利用这一点
 - 传递的参数为`dictobj.items()`, 排序后的元素为元组(`key, value`)
 - 可迭代对象中的元素并没有任何限制, 可以是任何类型的对象
 - 如果对象间无法比较大小, 则需要通过`key`参数传递一个提供排序基准的函数对象
- 怎么排序? 即排序的基准是什么?
 - 可以利用元素本身的大小关系
 - 有多个排序基准时, 计算排序基准的函数对象返回一个包含(主排序基准, 第2排序基准, ...)的元组对象
 - 在多个排序基准中, 如果升序和降序不统一
 - 某个排序基准是数值类型, 则可以利用负数运算实现排序基准的统一

内置函数sorted示例

单次销售信息为水果名称、价格和数量

```
>>> sales = [['apple', 6.0, 5], ['banana', 2.30, 10], ['apple', 5.50, 10], ['orange', 4.50, 5],  
             ['pearl', 4.50, 4.5], ['banana', 2.30, 5], ['pearl', 5.50, 10]]
```

基于名称进行排序

排序基准为名称。名称相同时按照原有顺序排序

```
>>> sorted(sales, key=lambda item:item[0])  
[['apple', 6.0, 5], ['apple', 5.5, 10], ['banana', 2.3, 10], ['banana', 2.3, 5], ['orange', 4.5, 5],  
 ['pearl', 4.5, 4.5], ['pearl', 5.5, 10]]
```

```
>>> sorted(sales)  
[['apple', 5.5, 10], ['apple', 6.0, 5], ['banana', 2.3, 5], ['banana', 2.3, 10], ['orange', 4.5, 5],  
 ['pearl', 4.5, 4.5], ['pearl', 5.5, 10]]
```

- 基于元素之间的大小关系进行排序：有序序列的比较
- 基于名称排序，名称相同时比较价格，价格相同时比较数量

内置函数sorted示例

单次销售信息为水果名称、价格和数量

```
>>> sales = [['apple', 6.0, 5], ['banana', 2.30, 10], ['apple', 5.50, 10], ['orange', 4.50, 5],  
             ['pearl', 4.50, 4.5], ['banana', 2.30, 5], ['pearl', 5.50, 10]]
```

基于价格排序, 价格相同时, 基于数量排序, 数量相同时基于水果名排序

```
>>> sorted(sales, key=lambda item:(item[1], item[2], item[0]))  
[['banana', 2.3, 5], ['banana', 2.3, 10], ['pearl', 4.5, 4.5], ['orange', 4.5, 5], ['apple', 5.5,  
10], ['pearl', 5.5, 10], ['apple', 6.0, 5]]
```

有多个排序基准时, 计算排序基准的函数对象返回一个包含(主排序基准, 第2排序基准, ...)的元组对象:

- 注意lambda表达式中的圆括号不能省略
- 如果省略, `sorted(sales, key=lambda item:item[1], item[2], item[0])`
 - 解释为关键字参数key和两个位置参数。 `SyntaxError: positional argument follows keyword argument`

基于价格排序, 价格相同时, 基于数量排序, 数量相同时基于水果名。数量要求采用降序, 其余升序排列

```
>>> sorted(sales, key=lambda item:(item[1], -item[2], item[0]))  
# [['banana', 2.3, 10], ['banana', 2.3, 5], ['orange', 4.5, 5], ['pearl', 4.5, 4.5],  
['apple', 5.5, 10], ['pearl', 5.5, 10], ['apple', 6.0, 5]]
```

在多个排序基准中, 如果升序和降序不统一

- 某个排序基准是数值类型, 可利用负数运算实现排序基准的统一: $a < b \Leftrightarrow -a > -b$

内置函数sorted示例

- 传递的参数为字典，则新列表中的元素就是字典的key
 - 字典对象通过key可以得到其对应的value，sorted传递的key参数可以利用这一点
- 传递的参数为dictobj.items()，排序后的元素为元组(key, value)

记录水果库存: key=水果名, value=库存

```
>>> inventory = {'orange': 45, 'pear': 58, 'apple': 125, 'banana': 45, 'peach': 15}
```

基于水果名排序

排序基准: 水果名。字典对象作为可迭代对象时其元素为key，基于元素本身的大小关系排序

```
>>> [(fruit, inventory[fruit]) for fruit in sorted(inventory)]  
[('apple', 125), ('banana', 45), ('orange', 45), ('peach', 15), ('pear', 58)]
```

```
>>> sorted(inventory.items())  
[('apple', 125), ('banana', 45), ('orange', 45), ('peach', 15), ('pear', 58)]
```

基于数量排序, 数量相同时基于水果名排序

排序基准: (水果数量, 水果名)

```
>>> [(fruit, inventory[fruit]) for fruit in sorted(inventory, key=lambda fruit: (inventory[fruit], fruit))]  
[('peach', 15), ('banana', 45), ('orange', 45), ('pear', 58), ('apple', 125)]
```

```
>>> sorted(inventory.items(), key=lambda item: (item[1], item[0]))  
[('peach', 15), ('banana', 45), ('orange', 45), ('pear', 58), ('apple', 125)]
```

内置函数sorted示例

基于最高温度对城市名称进行排序，返回城市的名称的列表

- 根据城市名称得到对应的下标，相应的得到对应的最高温度
- 两个列表组合后排序，然后再得到对应的城市名称

```
>>> cities = ['Shanghai', 'Beijing', 'Wuhan', 'Chengdu', 'Nanjing']
>>> temperatures = [30, 26, 34, 29, 38]

>>> sorted(cities, key=lambda city:temperatures[cities.index(city)])
['Beijing', 'Chengdu', 'Shanghai', 'Wuhan', 'Nanjing']

>>> [city for temp, city in sorted(zip(temperatures, cities))]
['Beijing', 'Chengdu', 'Shanghai', 'Wuhan', 'Nanjing']
```

0	1	2	3	4
Shanghai	Beijing	Wuhan	Chengdu	Nanjing
30	26	34	29	38

zip(temperatures,cities)

30,	Shanghai
26,	Beijing
...	

functools.cmp_to_key

- `functools`模块的`cmp_to_key(cmp)`: 将一个比较函数对象`cmp`转换为可以作为排序基准的函数对象
- 基于排序基准比较两个元素`e1`和`e2`的大小, 最终调用`cmp(e1, e2)`, 根据其返回值决定`e1`和`e2`的顺序

`e1 < e2`, if `cmp(e1, e2) < 0`, 一般为-1
`e1 == e2`, if `cmp(e1, e2) == 0`
`e1 > e2`, if `cmp(e1, e2) > 0`, 一般为1

```
records = [['tony', 'math', 100], ['tom', 'math', 99], ['bob', 'cs', 98], ['tony', 'cs', 88], ['tony', 'ee', 87]]  
# 姓名升序排列, 姓名相同的时候, 根据专业进行降序排列
```

```
def my_cmp_person(p1, p2):  
    if p1[0] > p2[0]:  
        return 1  
    elif p1[0] < p2[0]:  
        return -1  
    else:  
        if p1[1] > p2[1]:  
            return -1  
        elif p1[1] < p2[1]:  
            return 1  
        else:  
            return 0
```

sorted_cmp.py



`key(e1) <, ==, > key(e2)?`

`cmp(e1, e2) == -1, 0, 1`

```
k = sorted(records, key=functools.cmp_to_key(my_cmp_person))
```

```
[['bob', 'cs', 98], ['tom', 'math', 99], ['tony', 'math', 100], ['tony', 'ee', 87], ['tony', 'cs', 88]]
```

recap 用于序列操作的常用内置函数(max和min)

max和min: 两种语法

- `max(arg1, arg2, *args, key=func)`、`min(arg1, arg2, *args, key=func)`: 传递的参数中的最大或最小值
- `max(iterable, default=obj, key=func)`、`min(iterable, default=obj, key=func)`: 返回迭代对象中的**最大或最小元素**, 如果为空, 返回obj
- `key`指出如何比较大小, 即**基于调用函数`key(item)`的大小**来判断。缺省比较参数或者元素的大小
- 传递的参数或者可迭代对象的元素并不要求一定是数值类型, 只要可以按照相应的基准比较大小即可

```
>>> s = [1, 4, -5, -7, 6]
>>> max(*s)
6
>>> min(s)
-7
>>> max(s, key=abs)
-7
>>> min(*s, key=abs)
1

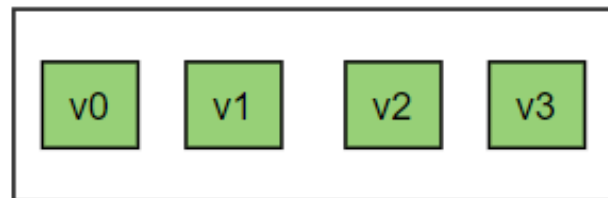
>>> words = ['nice', 'to', 'console']
>>> max(words, key=len)
'console'
```

主要内容

- 字典的定义和字典序列解包
- 字典的方法
- sorted函数
- **集合**

集合(set)

- 集合是无序可变对象，也是一个可迭代对象
 - 集合中的元素必须是可hash对象，不能是可变对象
 - 集合中的元素不重复，且无序（内部的顺序，不保留插入顺序）
- 集合字面量：
 - 使用一对大括号界定，元素间以逗号分隔
 - 定义时可以包含重复的对象，但只会保留一个
- 构造函数：set([iterable])
 - 创建一个set对象，其元素为iterable对象中的元素。类似的，重复的元素仅保留一个
 - {}定义的是空字典，而不是空集合，set()创建空集合对象
- 集合对象的copy方法：原有集合对象的克隆版本，等价于set(s)
- 集合推导式：类似于列表推导式，可以有多个for子句，{expr for item in iterable if expr}



```
>>> {3, 7, 5, 5}
{3, 5, 7}
>>> {3, [1, 2]}
...
TypeError: unhashable type: 'list'
>>> type({})
<class 'dict'>
>>> set()
set()
>>> set(range(8, 14))
{8, 9, 10, 11, 12, 13}
```

```
>>> set([0, 1, 0, 1, 7, 8])
{0, 1, 7, 8}
>>> s = {2*x for x in range(5)}
>>> s
{0, 2, 4, 6, 8}
>>> len(s)
5
>>> s.copy()
{0, 2, 4, 6, 8}
```

集合的运算

整数的位运算符: 逐个二进制比特运算

- 或运算 | 全0才为0, 否则为1
- 与运算 & 全1才为1, 否则为0
- 异或运算 ^ 半加, 相等为0, 不等为1

- 表格中的运算符或方法不是原地修改, 返回一个新的集合对象

方法	说明	例子
$s1 \mid s2 \mid \dots$ <code>s1.union(s2,...)</code>	• 并集, 在任一集合, $s1 \cup s2 \cup \dots$ • $s1 \cup \text{set}(s2) \cup \dots$	$\{1,2,3\} \mid \{2,4\} = \{1,2,3,4\}$
$s1 \& s2 \& \dots$ <code>s1.intersection(s2,...)</code>	• 交集, 在所有集合出现, $s1 \cap s2 \cap \dots$ • $s1 \cap \text{set}(s2) \cap \dots$	$\{1,2,3\} \& \{2,4\} = \{2\}$
$s1 - s2$ <code>s1.difference(s2)</code>	• 差集, 在s1但不在 s2, $s1 - s2$ • $s1 - \text{set}(s2)$	$\{1,2,3\} - \{2,4\} = \{1,3\}$
$s1 \wedge s2$ <code>s1.symmetric_difference(s2)</code>	对称差集, $s1 \oplus s2 = (s1 \cup s2) - (s1 \cap s2)$ 独属于其中某个集合, 即只属于其中一个但不属于另一个	$\{1,2,3\} \wedge \{2,4\} = \{1, 3, 4\}$

```
>>> s1 = {1, 2, 3}
>>> s2 = {2, 4}
>>> s3 = {2, 5}
>>> s1 | s2
{1, 2, 3, 4}
>>> s1.union(s2, s3)
{1, 2, 3, 4, 5}
>>> s1.intersection(s2, s3)
{2}
```

```
>>> s1 - s2
{1, 3}
>>> s2.difference(s1) # s2 - s1
{4}
>>> s1 ^ s2
{1, 3, 4}
>>> s1.symmetric_difference(s2)
{1, 3, 4}
```

集合的运算

- 判断集合之间的关系

方法	说明
<code>s1.isdisjoint(s2)</code>	$s1 \cap s2 == \phi$, 即没有共同元素为True
<code>s1.issubset(s2)</code> <code>s1 <= s2</code> <code>s1 < s2</code>	$s1 \subseteq s2$, s1是否为s2的子集 $s1 \subseteq s2$ $s1 \subset s2$ s1是否为s2的真子集
<code>s1.issuperset(s2)</code> <code>s1 >= s2</code> <code>s1 > s2</code>	$s1 \supseteq s2$, s1是否为s2的超集 $s1 \supseteq s2$ $s1 \supset s2$, s1是否为s2的真超集
<code>s1 == s2</code> <code>s1 != s2</code>	判断两个集合是否相等
<code>value in s</code> <code>value not in s</code>	$value \in s$, 判断集合中是否有value $value \notin s$

```
>>> s1 = {1, 2, 3}
>>> s2 = {3, 5}
>>> s3 = {5, 6}
>>> s4 = {2, 3}
>>> s1.isdisjoint(s2)
False
>>> s1.isdisjoint(s3)
True
>>> s4.issubset(s1)
True
>>> s4 <= s1
True
>>> s4 < s1
True
>>> s4 < s2
False
>>> s1.issuperset(s4)
True
>>> s3 > s2
False
>>> s1 != s2
True
>>> 1 in s1
True
>>> 1 not in s2
True
```

集合的原地修改

s1 = {1, 2, 3}
s2 = {2, 3, 4}

方法	说明	例子
s.add(x)	将x添加到集合s	s1.add('a') # s1 = {1,2,3,'a'}
s1.update(s2,...)	并集 s1 = s1 $\cup$ set(s2) $\cup$...	s1.update(s2) # s1 = {1,2,3,4}
s1.intersection_update(s2,...)	交集 s1 = s1 $\cap$ set(s2) $\cap$...	s1.intersection_update(s2) # s1 = {2,3}
s1.difference_update(s2)	差集 s1 = s1 - set(s2)	s1.difference_update(s2) # s1 = {1}
s1.symmetric_difference_update(s2)	对称差集 s1 = s1 $\oplus$ set(s2)	s1.symmetric_difference_update(s2) # s1 = {1,4}
s.remove(x)	从集合s中移除x, 不存在抛异常KeyError	s1.remove(1) # s1 = {2,3}
s.discard(x)	从集合s中移除x	s1.discard(1) # s1 = {2,3}
s.pop()	从集合s移除一个元素并返回, 空时KeyError	s1.pop() # return 1, s1 = {2,3}
s.clear()	清空集合s	s1.clear() # s1为空集合

```
>>> s1 = {1, 2, 3}
>>> s1.add(4)
>>> s1.update(range(2))
>>> s1
{0, 1, 2, 3, 4}
```

```
>>> s1.remove(0)
>>> s1.remove(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 0
```

```
>>> s1.discard(0)
>>> s1
{1, 2, 3, 4}
>>> s1.pop()
1
```

集合示例1

- 集合示例1：判断是否有重复的元素

duplicated_items.py

```
import random

def is_duplicated(s):
    return len(set(s)) != len(s)

if __name__ == "__main__":
    s = [random.randint(0, 100) for i in range(20)]
    print(s)
    if is_duplicated(s):
        print("duplicated items!")
    else:
        print("no duplicated items!")
```

集合示例2：去除重复元素

```
>>> s = [0, 11, 8, 7, 9, 10, 2, 3, 3, 13]
>>> list(set(s))
[0, 2, 3, 7, 8, 9, 10, 11, 13]
```

```
def unique_list(s):
    uniqs = []
    for item in s:
        if item not in uniqs:
            uniqs.append(item)
    return uniqs
```

```
def unique_list2(s):
    # return list(set(s))
    # 如果要保留之前的顺序
    return sorted(set(s), key=s.index)
```

```
def unique_list3(s):
    d = dict.fromkeys(s)
    return list(d)
```

```
def unique_list4(s):
    import collections
    d = collections.OrderedDict.fromkeys(s)
    return list(d)
```

unique_list.py

```
if __name__ == '__main__':
    # 产生10个[0,20)之间的随机整数, 保存在列表中
    # s = [random.choice(range(20)) for i in range(10)]
    s = random.choices(range(20), k=10)
    print(*s)
    print(*unique_list2(s))
```

- 利用**set不重复特性去除重复元素**, 但是:
 - **集合实现并不保留插入顺序**, 而是内部的顺序
 - 无序特性导致产生的list不是按照原来的顺序
 - 按照原来的顺序: 基于集合中的元素在原有列表的(首次)出现顺序, 即**元素在原有列表的首次出现的下标**, 即s.index(element)
- 利用**dict的key的不重复特性去除重复元素**
 - Python3.6以后的dict实现是按照key插入的顺序
 - Python3.6之前使用collections.OrderedDict保留原来的顺序
- **课堂思考**: 实现函数my_sample(seq, k), 完成random.sample类似的功能, 从seq中不放回取k个元素

集合示例3：集合的运算(交集和差集)

函数check_options: (1) options给出了调用时传递的选项和相应的取值。(2) 支持选项verbose/reverse/key

parse_kwargs.py

```
def check_options(**kwargs):
    valid_options = {'verbose', 'reverse', 'key'}
    passed_options = valid_options.intersection(kwargs)
    # passed_options = valid_options & set(kwargs)
    print('passed options:', *passed_options)
    unknown_options = set(kwargs) - valid_options
    if unknown_options: # len(unknown_options)
        print('unknown options:', *unknown_options)

if __name__ == '__main__':
    options = {'reverse':True, 'key':lambda item:item[0], 'option1':1, 'option2':2}
    check_options(**options)
```

集合A - 集合B = {在集合A但不在集合B中的元素}

集合示例4

问题13-14: 编写函数，接收任意多个实数，返回一个元组，其中第一个元素为所有参数的平均值，其他元素为所有参数中大于平均值的实数

greater_average.py

```
def greater_average(*args):
    if not args:
        raise ValueError("参数不能为空")
    if any(not isinstance(v, (bool, int, float)) for v in args):
        # if not {type(v) for v in args} <= {bool, int, float}:
        raise ValueError("参数必须是数值类型")

    average = sum(args) / len(args)
    greater = [i for i in args if i > average]
    return average, *greater
    # return (average,) + tuple(greater)

if __name__ == "__main__":
    print(greater_average(1, 2, 3, 4))
```